Chapter 8 - Classification and Generalized regression: Exercise solutions

Mattias Villani

Click on the arrow to see a solution.

Exercise 8.1

The dataset `runningtime.csv` available at https://github.com/mattiasvillani/BayesianLearningBook/raw/main/data/runningtime.csv contains measurements on the running time (`time`) for a computer code on $n = 100$ computers with different hardware specifications and operating systems. The hardware performance is summarized by a normalized performance score (`performance`), where a lower score means better performance. The operating system is either Windows (`os=1`) or Linux (`os=0`). Consider the following exponential regression model

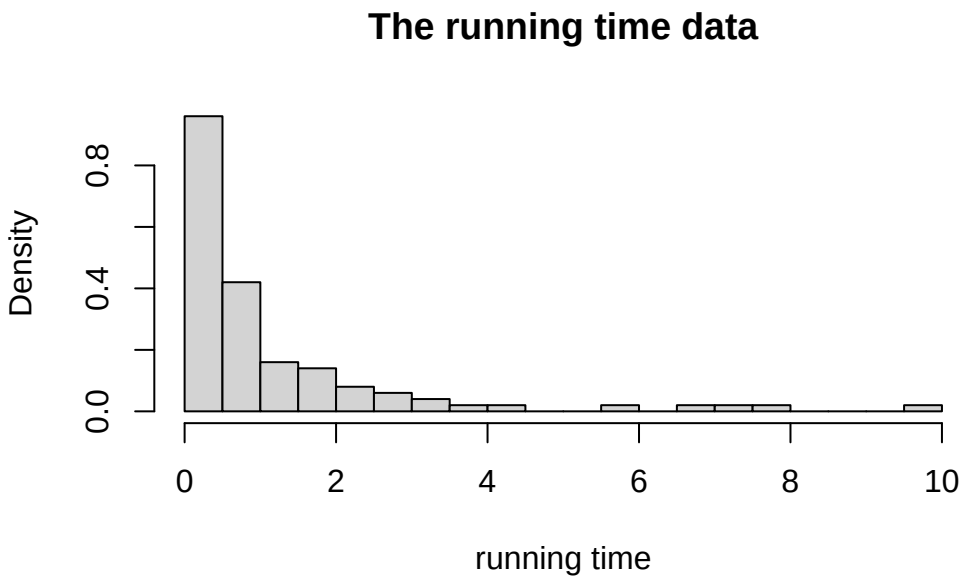
for $i = 1, \ldots, n$

$$y_i|\mathbf{x}_i, \beta \sim \text{Expon}\left(\frac{1}{\exp\left(\mathbf{x}_i^\top \beta\right)}\right)$$

where y_i is the response variable, $\mathbf{x}$ is a vector with three elements, the first element is 1 for the intercept, the second element is the performance measure and the final element if the operating system for the computer. Note that $\mathbb{E}(y_i|\mathbf{x}_i) = \exp\left(\mathbf{x}_i^\top \beta\right)$ for this model. Let the prior be $\beta \sim N(0, \tau^2 I)$, where $\tau = 10$ and I is the identity matrix.

Loading the data and mvtnorm package

```
library(mvtnorm)
data = read.csv("https://raw.githubusercontent.com/mattiasvillani/BayesianLearningBook/main/d
y = data$time
X = cbind(1, data$performance, data$os)
hist(y, 30, freq = FALSE, main = "The running time data", xlab = "running time")
```



a) Do a normal approximation of the posterior distribution $p(\beta|\mathbf{y}, \mathbf{X})$ based on numerical optimization. Simulate 10,000 samples from this approximation and make histograms to approximate the marginal posterior for each regression coefficient in β.

Solution

We will use optim to get the posterior mode and posterior observed information matrix for the normal approximation. The log posterior function is

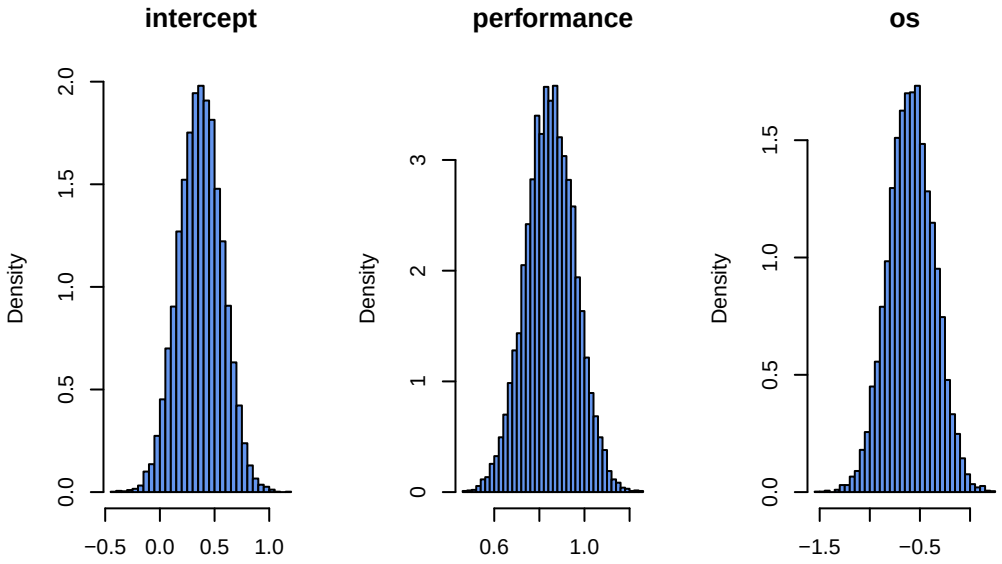
```
logPostExpon <- function(betaVect, y, X, tau){
  p <- length(betaVect)
  linPred <- X%*%betaVect
  logLik <- sum(dexp(y, rate = 1/exp(linPred), log = TRUE) )
  logPrior <- dmvnorm(betaVect, rep(0, p), tau^2*diag(p), log=TRUE)
  return(logLik + logPrior)
}
```

We use optim to find the posterior mode and posterior information matrix:

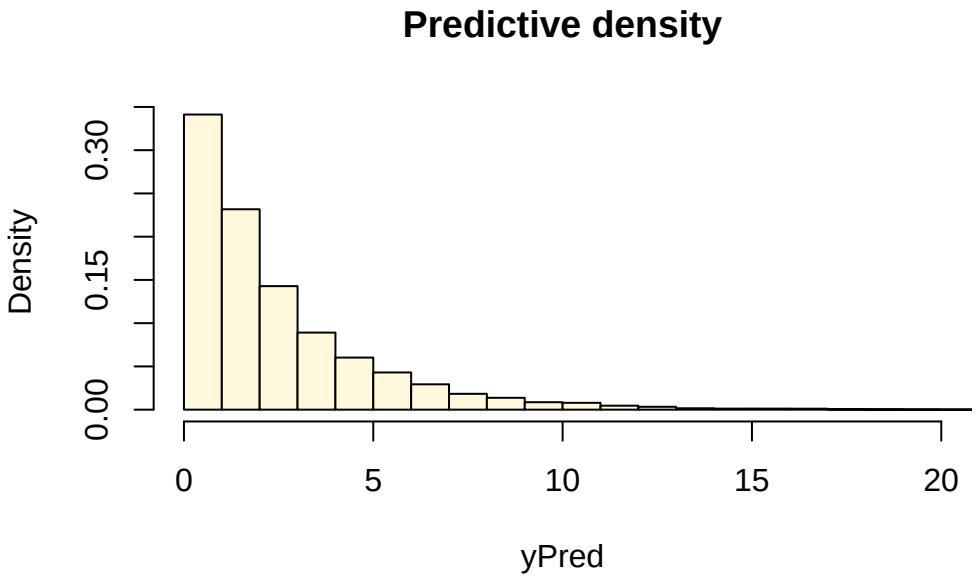
```
p = dim(X)[2]
tau = 10
initVal <- as.vector(rep(0,p))
OptimResults<-optim(initVal, logPostExpon,gr=NULL, y, X, tau,
  method=c("BFGS"), control=list(fnscale=-1), hessian=TRUE)
postMode = OptimResults$par
postCov = -solve(OptimResults$hessian) # inv(J) - Approx posterior covariance matrix
postStd = sqrt(diag(postCov))
```

Simulating from the normal approximation and plotting histograms of the marginal posteriors:

```
nSim = 10000
library(mvtnorm)
betaDraws = rmvnorm(nSim, postMode, postCov)
par(mfrow = c(1,p))
varNames = c("intercept", "performance", "os")
for (j in 1:p){
hist(betaDraws[,j], 30, freq = FALSE, main = varNames[j],
     xlab = "", col = "cornflowerblue")
}
```



b) Use the normal approximation from a) and simulation to approximate the predictive distribution of the running time for a new Windows computer with performance measure equal to 1.3.

Solution

```
xPred = c(1, 1.3, 1)
yPredDraws = rep(0, nSim)
for (i in 1:nSim){
  yPredDraws[i] = rexp(1, 1/exp(xPred %*% betaDraws[i,]))
}
hist(yPredDraws, 30, freq = FALSE, main = "Predictive density",
     xlab = "yPred", col = "cornsilk", xlim = c(0,20))
```